FATIMA JINNAH WOMEN UNIVERSITY

Rawalpindi

DevOps – Lab 04

Database Environments and Credentials Management with Docker

Name	Mahnoor Sajjad
Roll No.	2023-BCS-054 (Section A)
Program	BS Computer Science
Course	DevOps
Lab	Lab 04 (Database Environments & Credentials)
GitHub Username	mahnoor-cs6767
Repository	https://github.com/mahnoor-cs6767/Databases

Objective

In this lab, you fork and clone a repository of database examples, confirm that Git and Docker are installed, check that four database containers (one MySQL and three MongoDB) are running, and then connect to them from a Python script. You start with connection details written directly in the code, then move to environment-based configuration (dev, test and prod), and finally store connection details in a protected credentials file.

Tools and technologies

- Git and GitHub CLI (gh) for forking, cloning and pushing code
- Docker and Docker Compose to run the database containers
- MySQL 8.4 and MongoDB 8.0 as the databases
- Python (virtual environment) with the db_connect.py test script

Lab overview

Step	Task	Command / tool
1	Fork the repository and log in to GitHub	gh auth login
2	Clone the forked repository	gh repo clone
3	Verify Git, Docker and Docker Compose	--version
4	Check the running database containers	docker compose ps
5	Confirm the port mappings	docker compose port
6	Connect to MySQL (hardcoded settings)	python db_connect.py
7	Connect to MongoDB (hardcoded settings)	python db_connect.py
8	Connect to MongoDB in dev, test and prod	python db_connect.py
9	Create the credentials file	cp -n
10	Restrict the credentials file permissions	chmod 600
11	Connect using the credentials file	export APP_ENV=...

Step 1: Fork the repository and log in to GitHub

Create your own copy of the instructor's Databases repository on GitHub, and sign in to GitHub from the terminal so you can push and pull code.

What to do

1. On GitHub, fork the **`WaqasSaleem97/Databases`** repository into your own account. It then appears in your repository list as "Forked from WaqasSaleem97/Databases".
2. In the terminal, log in with the GitHub CLI using the web flow:

```
gh auth login -w -s repo
```

3. Push your local DevOps work to GitHub to confirm that your login works:

```
git push origin main
```

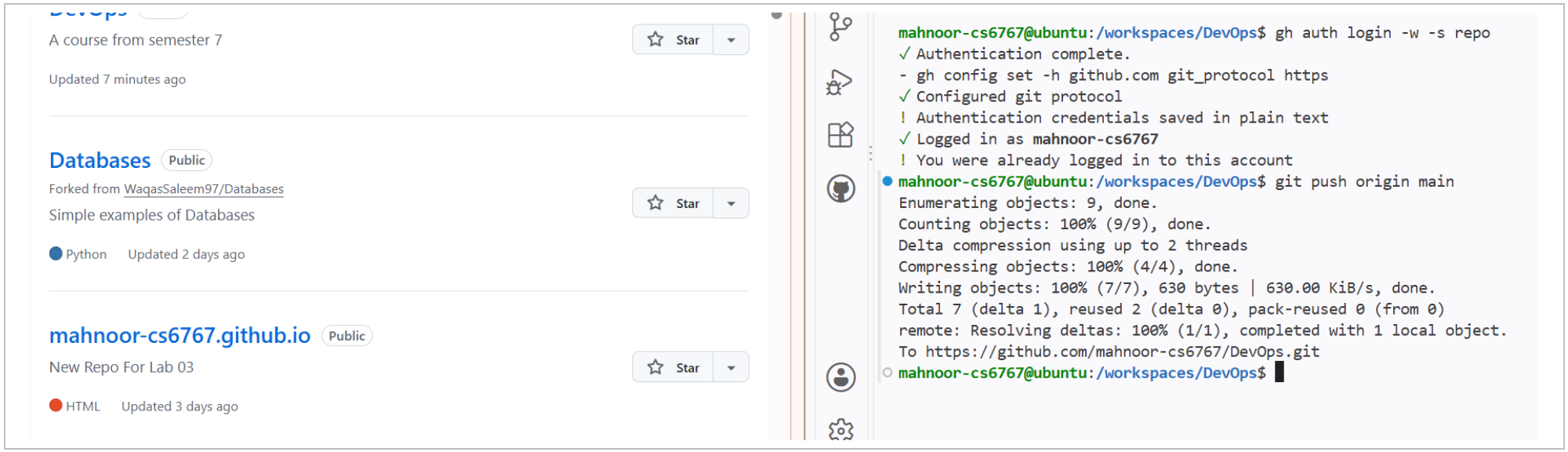


Figure 1: Forked Databases repository on GitHub (left) and GitHub CLI login with a successful push (right)

What to look for

- The Databases repository appears under your account, marked as forked from WaqasSaleem97/Databases.
- The terminal shows "Authentication complete" and "Logged in as" followed by your username.

- git push finishes with “Writing objects: 100%” and prints your repository address.

Step 2: Clone the forked repository

Download your fork to the workspace so you can work on the database files locally.

What to do

1. Move to the workspaces folder, then clone your fork and enter the new folder:

```
cd /workspaces
gh repo clone mahnoor-cs6767/Databases
cd Databases
```

```
mahnoor-cs6767@ubuntu:/workspaces/DevOps$ cd /workspaces
mahnoor-cs6767@ubuntu:/workspaces$ gh repo clone mahnoor-cs6767/Databases
Cloning into 'Databases'...
remote: Enumerating objects: 101, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 101 (delta 30), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (101/101), 39.86 KiB | 5.69 MiB/s, done.
Resolving deltas: 100% (30/30), done.
From https://github.com/WaqasSaleem97/Databases
 * [new branch]      main       -> upstream/main
! Repository WaqasSaleem97/Databases set as the default repository. To learn more about the default repository, run: gh repo s
t-default --help
mahnoor-cs6767@ubuntu:/workspaces$ cd Databases
mahnoor-cs6767@ubuntu:/workspaces/Databases$
```

Figure 2: Cloning the Databases repository with GitHub CLI

What to look for

- The clone finishes with all 101 objects received (100%).
- GitHub CLI also adds the original repository as an upstream remote (upstream/main), so you can pull future updates from the instructor.
- The prompt changes to /workspaces/Databases, confirming you are inside the project folder.

Step 3: Verify Git, Docker and Docker Compose

Before starting the databases, check that the required tools are installed and working.

What to do

1. Run each of these commands from the Databases folder:

```
git --version
docker --version
docker compose version
```

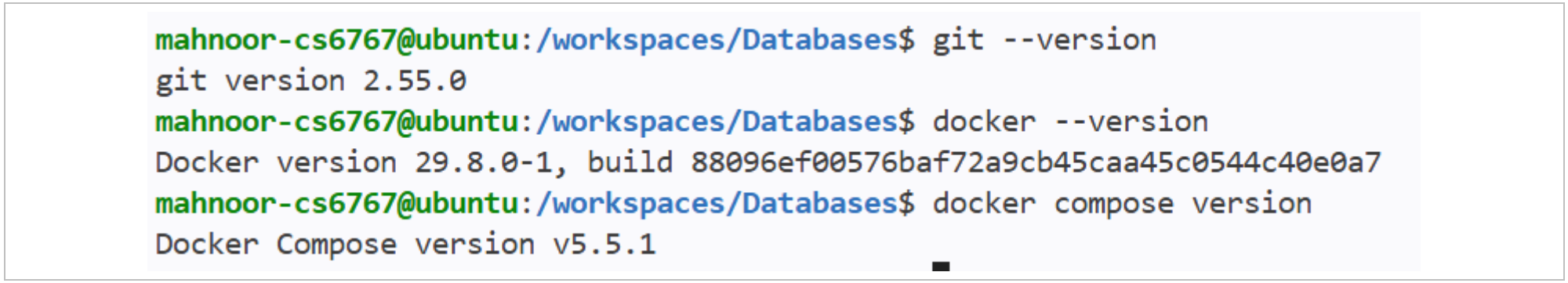


Figure 3: Versions of Git, Docker and Docker Compose

What to look for

Each command prints a version number instead of an error: Git 2.55.0, Docker 29.8.0 and Docker Compose v5.5.1.

Step 4: Check that the database containers are running

The project defines four database services in Docker Compose. List them to confirm that they are all up and healthy.

What to do

1. Run:

```
docker compose ps
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ docker compose ps
NAME                      IMAGE       COMMAND                  SERVICE      CREATED              STATUS                          PORTS
databases-mongo-dev-1     mongo:8.0   "docker-entrypoint.s…"   mongo-dev    About a minute ago   Up About a minute (healthy)     127.0.0.1:27017->27017/tcp
databases-mongo-prod-1    mongo:8.0   "docker-entrypoint.s…"   mongo-prod   About a minute ago   Up About a minute (healthy)     127.0.0.1:27019->27017/tcp
databases-mongo-test-1    mongo:8.0   "docker-entrypoint.s…"   mongo-test   About a minute ago   Up About a minute (healthy)     127.0.0.1:27018->27017/tcp
databases-mysql-1         mysql:8.4   "docker-entrypoint.s…"   mysql        About a minute ago   Up About a minute (healthy)     33060/tcp, 127.0.0.1:3307->3306/tcp
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$
```

Figure 4: Four database containers running and healthy

What to look for

Container	Image	Service	Port mapping
databases-mongo-dev-1	mongo:8.0	mongo-dev	127.0.0.1:27017 → 27017
databases-mongo-test-1	mongo:8.0	mongo-test	127.0.0.1:27018 → 27017
databases-mongo-prod-1	mongo:8.0	mongo-prod	127.0.0.1:27019 → 27017
databases-mysql-1	mysql:8.4	mysql	127.0.0.1:3307 → 3306

- The STATUS column shows “Up … (healthy)” for every container.
- The prompt starts with (.venv), which means the Python virtual environment is active.

Step 5: Confirm the port mappings

Each container listens on its standard port inside Docker, but is published to a different port on your machine. This command asks Docker Compose which host port each service uses.

What to do

1. Run the following (it checks all four services in one line):

```
docker compose port mysql 3306 && docker compose port mongo-dev 27017 && docker compose port mongo-test 27017 && docker compose port mongo-prod 27017
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ docker compose port mysql 3306 && docker compose port mongo-dev 27017 && docker compose port mongo-test 27017
 && docker compose port mongo-prod 27017
127.0.0.1:3307
127.0.0.1:27017
127.0.0.1:27018
127.0.0.1:27019
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$
```

Figure 5: Host ports for MySQL and the three MongoDB services

What to look for

The four lines of output match the mapping in Step 4, in the same order as the command: MySQL on 3307, MongoDB dev on 27017, test on 27018 and prod on 27019. All of them are bound to 127.0.0.1, so they can only be reached from your own machine.

Step 6: Connect to MySQL with hardcoded settings

Run the Python test script with the MySQL connection details written directly in the code. This is the simplest approach and serves as the starting point for the lab.

What to do

1. Run:

```
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$
```

Figure 6: Successful MySQL connection using hardcoded settings

What to look for

- "Connected to MySQL: school_mysql" confirms the connection works.
- The server is 127.0.0.1 on port 3307, the published MySQL port.
- The environment is reported as dev.

Step 7: Connect to MongoDB with hardcoded settings

Repeat the test for MongoDB, again with the connection details fixed inside the script.

What to do

1. Run:

```
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$
```

Figure 7: Successful MongoDB connection using hardcoded settings

What to look for

- "Connected to MongoDB: school_dev" confirms the connection works.
- The server is 127.0.0.1 on port 27017, the MongoDB dev container.
- Because the settings are fixed in the code, you would have to edit the script every time you wanted to use the test or prod database. The next steps remove that limitation.

Step 8: Connect to MongoDB in the dev, test and prod environments

Now the same script is run against each of the three MongoDB environments. The code stays the same; only the environment setting changes, and the script connects to a different database and port each time.

What to do

1. Select the dev environment and run the script:

```
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
```

Figure 8: Dev environment: school_dev on port 27017

2. Select the test environment and run the script again:

```
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
```

Figure 9: Test environment: school_test on port 27018

3. Select the prod environment and run the script once more:

```
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
```

Figure 10: Prod environment: school_prod on port 27019

What to look for

Environment	Database	Port	Container
dev	school_dev	27017	mongo-dev
test	school_test	27018	mongo-test
prod	school_prod	27019	mongo-prod

The “Environment” line, the database name and the port all change together, which shows that the script picks its settings from the selected environment.

Step 9: Create the credentials file

Instead of keeping connection details in the code, they can be stored in a separate credentials file. The repository includes a template, credentials.ini.example, which you copy to create your own credentials.ini.

What to do

1. Copy the template without overwriting an existing file, then check that the new file exists:

```
cp -n credentials.ini.example credentials.ini
ls credentials.ini
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ cp -n credentials.ini.example credentials.ini
cp: warning: behavior of -n is non-portable and may change in future; use --update=none instead
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ ls credentials.ini
credentials.ini
```

Figure 11: Creating credentials.ini from the example template

What to look for

- ls prints credentials.ini, so the file was created.
- The warning about -n being "non-portable" is harmless. It only says that a newer option, --update=none, is preferred, and the copy still worked.

Step 10: Restrict the credentials file permissions

A credentials file should only be readable by its owner. Setting the permissions to 600 gives the owner read and write access and gives everyone else no access.

What to do

1. Change the permissions, then display them to verify:

```
chmod 600 credentials.ini
stat -c '%a %n' credentials.ini
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ chmod 600 credentials.ini
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ stat -c '%a %n' credentials.ini
600 credentials.ini
```

Figure 12: Setting and verifying 600 permissions on credentials.ini

What to look for

The stat command prints 600 credentials.ini, confirming the permissions were applied.

Step 11: Connect using the credentials file for each environment

Finally, run the script with the connection details read from the credentials file. The APP_ENV variable tells the script which environment section to use.

What to do

1. Set APP_ENV to dev and run the script:

```
export APP_ENV=dev
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ export APP_ENV=dev
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
```

Figure 13: Dev environment using the credentials file

2. Set APP_ENV to test and run the script:

```
export APP_ENV=test
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ export APP_ENV=test
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
```

Figure 14: Test environment using the credentials file

3. Set APP_ENV to prod and run the script:

```
export APP_ENV=prod
python db_connect.py
```

```
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ export APP_ENV=prod
(.venv) mahnoor-cs6767@ubuntu:/workspaces/Databases$ python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
```

Figure 15: Prod environment using the credentials file

What to look for

Each run connects to the matching database (school_dev on 27017, school_test on 27018 and school_prod on 27019), and the Environment line matches the value you exported. Changing one variable is all it takes to switch environments, with no code changes.